

Unidad Didáctica

**Desarrollo de aplicaciones con
Node.js, Unidad II**

Contenido

Índice de Contenido	2
Introducción	4
Desarrollo de Contenidos	5
Configuración de un proyecto MEAN.....	5
Un vistazo a Express, Node y NPM.....	5
Creación de proyectos con Express	7
Modificando Express para MVC	8
Creación de sitios estáticos con Node.js y Express.....	11
Definición de rutas en Express.....	12
Creación de controladores básicos	14
Creación de vistas.....	16
Creación de modelo de datos con MongoDB y Mongoose.....	19
Conectando aplicaciones en Express a MongoDB usando Mongoose.....	19
Modelado de datos	19
Definiendo esquemas con Mongoose	22
Uso de la Shell de MongoDB para crear bases de datos y manipular información	23
Creación de APIs REST para exponer bases de datos de MongoDB a la aplicación	25
Configurando APIs en Express	25
Métodos GET, POST, PUT, DELETE	25

Consumo de APis REST de Express	28
Invocar APis desde Express	28
Use de datos obtenidos desde APis	29
Obtener documentos usando APis	31
Agregando datos a la base de datos por medio de APis	33
Protección de la integridad de los datos con validaciones de datos.....	34
Bibliografía y Webgrafía	37
Glosario	37



Bienvenidos a la Unidad II de nuestro curso, donde nos adentraremos en el emocionante mundo del desarrollo de aplicaciones utilizando Node.js. En esta etapa

del curso, nos centraremos en dominar las habilidades fundamentales necesarias para construir aplicaciones robustas y eficientes, haciendo uso de las tecnologías más relevantes en el panorama actual.

Durante esta unidad, exploraremos detalladamente cómo gestionar el manejo de dependencias y paquetes a través de package.json, configurar proyectos utilizando el popular framework Express con una arquitectura Modelo-Vista-Controlador (MVC), y cómo crear prototipos estáticos de aplicaciones dinámicas.

Además, nos sumergiremos en el fascinante mundo de las bases de datos NoSQL, centrándonos específicamente en MongoDB y su integración con Node.js mediante Mongoose. Aprenderemos a definir esquemas de datos, conectar aplicaciones Express a MongoDB, y crear APIs REST para interactuar con la base de datos de forma eficiente y segura.

En cada sección de esta unidad, nos enfocaremos en desarrollar un conjunto de habilidades específicas que te permitirán construir aplicaciones web modernas y escalables. Desde la configuración inicial del proyecto MEAN (MongoDB, Express, Angular, Node.js), hasta el consumo de APIs REST externas, cada paso será una oportunidad para expandir tu conocimiento y habilidades en el desarrollo de aplicaciones con Node.js.

¡Prepárate para sumergirte en un viaje emocionante hacia el dominio del desarrollo de aplicaciones con Node.js!

Configuración de un proyecto MEAN

En esta sección, daremos los primeros pasos hacia la creación de un proyecto MEAN, una pila tecnológica que integra MongoDB, Express.js, AngularJS (o Angular) y Node.js. Comenzaremos con un vistazo a las herramientas fundamentales que utilizaremos en nuestro desarrollo: Express, Node.js y NPM (Node Package Manager).

1. **Instalación de Node.js:** El primer paso es asegurarse de tener Node.js instalado en tu sistema. Puedes descargarlo e instalarlo desde el sitio web oficial de Node.js.
2. **Creación de un nuevo directorio:** Crea un directorio en tu sistema donde desees almacenar tu proyecto MEAN. Puedes hacerlo desde la línea de comandos utilizando el comando `mkdir` seguido del nombre del directorio.

```
mkdir nombre_proyecto_mean
```

Un vistazo a Express, Node y NPM

3. **Inicialización de un nuevo proyecto Node.js:** Abre una terminal, navega hasta el directorio recién creado y ejecuta el siguiente comando para inicializar un nuevo proyecto Node.js. Este comando creará un archivo `package.json`, que es esencial para administrar las dependencias del proyecto.

```
cd nombre_proyecto_mean  
npm init -y
```

4. **Instalación de Express.js:** Express.js es un framework web para Node.js. Puedes instalarlo ejecutando el siguiente comando en la terminal:

```
npm install express --save
```

El flag `--save` agrega `express` como una dependencia en el archivo `package.json`.

5. Creación de un archivo principal de la aplicación: Crea un archivo JavaScript (por ejemplo, `app.js` o `server.js`) en tu directorio del proyecto para escribir el código principal de tu aplicación Express.

```
// app.js

const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('¡Hola, mundo!');
});

app.listen(port, () => {
  console.log(`Servidor escuchando en
http://localhost:${port}`);
});
```

6. Ejecución de la aplicación: Para ejecutar tu aplicación, simplemente ejecuta el archivo principal de la aplicación desde la terminal utilizando Node.js.

```
node app.js
```

Esto iniciará el servidor Express y lo dejará escuchando en el puerto especificado (en este caso, el puerto 3000). Puedes acceder a tu aplicación abriendo un navegador web y navegando a `http://localhost:3000`.

7. Configuración de MongoDB: Para completar la configuración MEAN, necesitarás tener una instancia de MongoDB ejecutándose localmente o configurar una conexión a un servidor remoto de MongoDB. Puedes utilizar MongoDB Atlas para una solución de base de datos en la nube.

Con estos pasos, has configurado correctamente la parte de Node.js y Express.js de tu proyecto MEAN. Ahora puedes proceder a integrar MongoDB como tu base de datos y AngularJS o Angular como tu frontend, según tus necesidades específicas.

Creación de proyectos con Express

Para crear un proyecto con Express, sigue estos pasos:

1. Instalar Express: Si aún no lo has hecho, instala Express globalmente en tu sistema. Puedes hacerlo ejecutando el siguiente comando en tu terminal:

```
npm install -g express-generator
```

Este comando instalará el generador de aplicaciones Express, que te permitirá crear proyectos Express rápidamente.

2. Crear un nuevo proyecto Express: Una vez instalado el generador de Express, puedes crear un nuevo proyecto ejecutando el siguiente comando en tu terminal, seguido del nombre de tu proyecto:

```
express nombre_proyecto
```

Esto creará una estructura básica de archivos y directorios para tu proyecto Express dentro de un directorio llamado nombre_proyecto.

3. Navegar al directorio del proyecto: Después de crear el proyecto, navega al directorio del proyecto recién creado utilizando el comando cd en tu terminal:

```
cd nombre_proyecto
```

4. Instalar dependencias: Una vez dentro del directorio del proyecto, instala las dependencias del proyecto ejecutando el siguiente comando:

```
npm install
```

Esto instalará todas las dependencias listadas en el archivo package.json.

5. Ejecutar la aplicación: Una vez que las dependencias estén instaladas, puedes ejecutar tu aplicación Express utilizando el siguiente comando:

```
npm start
```

Este comando iniciará el servidor Express y lo dejará escuchando en el puerto especificado en el archivo app.js (por defecto, el puerto 3000).

6. Acceder a la aplicación: Abre un navegador web y navega a `http://localhost:3000` (o al puerto especificado si has modificado la configuración). Deberías ver la página de inicio predeterminada de Express, indicando que la aplicación se ha iniciado correctamente.

Con estos pasos, has creado un nuevo proyecto Express y has iniciado tu servidor web. Ahora puedes comenzar a desarrollar tu aplicación web utilizando Express y Node.js.

Modificando Express para MVC

Modificar Express para seguir el patrón de diseño Modelo-Vista-Controlador (MVC) implica organizar tu código en tres componentes principales: modelos, vistas y controladores. Aquí te explico cómo puedes hacerlo:

1. Creación de la estructura de directorios: Lo primero que necesitas hacer es crear una estructura de directorios que refleje el patrón MVC. Puedes organizar tus archivos de la siguiente manera:


```
nombre_proyecto/  
├── controllers/  
│   ├── mainController.js  
│   └── otroController.js  
├── models/  
│   └── modelo.js  
├── views/  
│   └── index.ejs  
├── public/  
│   ├── css/  
│   ├── js/  
│   └── img/  
└── app.js
```

En este ejemplo, controllers contendrá los controladores, models contendrá los modelos de datos, views contendrá las vistas y public contendrá recursos estáticos como archivos CSS, JavaScript e imágenes.

2. Configuración de Express para usar vistas: Para configurar Express para usar vistas, necesitas instalar un motor de plantillas. EJS (Embedded JavaScript) es un motor de plantillas popular y fácil de usar en Express. Puedes instalarlo ejecutando:

```
npm install ejs
```

Luego, configuramos Express en nuestro archivo app.js para usar EJS como motor de plantillas:

```
const express = require('express');  
const app = express();  
const port = 3000;  
  
// Configuración del motor de plantillas EJS  
app.set('view engine', 'ejs');  
app.set('views', './views');  
  
// Rutas y otros middlewares aquí...  
  
app.listen(port, () => {  
  console.log(`Servidor escuchando en  
http://localhost:${port}`);  
});
```

3. Crear controladores: En la carpeta controllers, crea archivos para tus controladores. Un controlador manejará las solicitudes HTTP y llamará a los modelos apropiados para acceder a los datos y luego renderizará las vistas.

Por ejemplo, mainController.js podría verse así:

```
// mainController.js
const mainController = {};

mainController.index = (req, res) => {
  res.render('index', { title: 'Página de inicio' });
};

module.exports = mainController;
```

4. Crear modelos: Los modelos en la carpeta models representan los datos de tu aplicación y proporcionan métodos para acceder y manipular esos datos.

Por ejemplo, modelo.js podría verse así:

```
// modelo.js
const Modelo = {};

Modelo.obtenerDatos = () => {
  // Lógica para obtener datos de la base de datos u otra fuente
  return ['dato1', 'dato2', 'dato3'];
};

module.exports = Modelo;
```

5. Crear vistas: Las vistas en la carpeta views contienen el HTML que se mostrará al usuario. Puedes usar el motor de plantillas EJS para incrustar código JavaScript en tus vistas. Por ejemplo, index.ejs podría verse así:

```
<!-- index.ejs -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title><%= title %></title>
</head>
<body>
  <h1><%= title %></h1>
  <!-- Otro contenido de la vista aquí -->
</body>
</html>
```

6. Configurar rutas en Express: En tu archivo app.js, configura las rutas para que apunten a los controladores correspondientes:

```
// app.js
const express = require('express');
const app = express();
const mainController = require('./controllers/mainController');

// Configuración del motor de plantillas EJS
app.set('view engine', 'ejs');
app.set('views', './views');

// Rutas
app.get('/', mainController.index);

app.listen(3000, () => {
  console.log('Servidor escuchando en el puerto 3000');
});
```

Con estos pasos, has configurado Express para seguir el patrón MVC. Ahora puedes comenzar a agregar lógica de controlador, acceso a datos en modelos y vistas a tu aplicación de forma organizada y escalable.

Creación de sitios estáticos con Node.js y Express

Para crear sitios estáticos con Node.js y Express y definir rutas, controladores básicos y vistas en un patrón de diseño Modelo-Vista-Controlador (MVC), puedes seguir los siguientes pasos:

1. Crear una estructura de directorios:

Crea una estructura de directorios para tu proyecto. Por ejemplo:

```
proyecto/
├── public/
│   ├── css/
│   ├── js/
│   └── img/
├── views/
│   ├── index.ejs
│   └── otra_vista.ejs
├── controllers/
│   └── controlador.js
└── app.js
```

Definición de rutas en Express

2. Instalar Express y EJS:

Si no has instalado Express previamente, instálalo en tu proyecto:

```
npm install express
```

También necesitarás EJS (Embedded JavaScript) como motor de plantillas para tus vistas:

```
npm install ejs
```

3. Configurar Express en app.js:

Importa Express y configúralo en tu archivo app.js:

```
const express = require('express');
const app = express();
const port = 3000;

app.set('view engine', 'ejs');
app.use(express.static('public'));

// Definir rutas
app.get('/', (req, res) => {
  res.render('index');
});

// Definir más rutas según sea necesario
```

```
app.listen(port, () => {  
  console.log(`Servidor escuchando en http://localhost:${port}`);  
});
```

4. Definir las vistas:

Crea archivos de vista dentro del directorio `views/`, utilizando el motor de plantillas EJS. Por ejemplo, `index.ejs`:

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Título de la página</title>  
  <link rel="stylesheet" href="/css/styles.css">  
</head>  
<body>  
  <h1>Bienvenido a mi sitio web</h1>  
</body>  
</html>
```

5. Crear controladores básicos:

Dentro del directorio `controllers/`, crea archivos para tus controladores.

Por ejemplo, `controlador.js`:

```
// controlador.js  
  
exports.index = (req, res) => {  
  res.render('index');  
};  
  
// Agregar más controladores según sea necesario
```

6. Definir rutas en Express:

En tu archivo app.js, importa y utiliza los controladores para definir las rutas:

```
const controlador = require('./controllers/controlador');  
  
app.get('/', controlador.index);  
  
// Agregar más rutas según sea necesario
```

Con estos pasos, has configurado un sitio estático utilizando Node.js y Express, con una estructura MVC básica. Ahora puedes ampliar tu aplicación agregando más rutas, controladores y vistas según tus necesidades.

Creación de controladores básicos

En el desarrollo de aplicaciones web utilizando Express, los controladores desempeñan un papel crucial en la organización y la lógica de negocio de tu aplicación. Los controladores son funciones que se encargan de manejar las solicitudes HTTP y generar respuestas apropiadas. Aquí te muestro cómo puedes crear controladores básicos en tu aplicación Express:

1. Crear un directorio para los controladores: En primer lugar, es una buena práctica organizar tus controladores en un directorio separado dentro de tu proyecto Express. Puedes llamar a este directorio "controllers". Puedes crearlo manualmente o utilizando la terminal:

```
mkdir controllers
```

2. Definir un controlador: Dentro del directorio de controladores, crea un nuevo archivo para tu controlador. Por ejemplo, podrías llamarlo userController.js. Este archivo contendrá las funciones que manejarán las solicitudes relacionadas con los usuarios en tu aplicación.
3. Escribir la lógica del controlador: En el archivo userController.js, escribe la lógica para manejar diferentes solicitudes relacionadas con los usuarios. Por

ejemplo, podrías tener una función para manejar la solicitud de la página de inicio de sesión y otra función para manejar la solicitud de registro de usuario. Aquí tienes un ejemplo básico:

```
// controllers/userController.js

// Función para manejar la solicitud de la página de inicio de sesión
exports.getLoginPage = (req, res) => {
  res.render('login'); // Renderiza la vista de inicio de sesión
};

// Función para manejar la solicitud de registro de usuario
exports.getRegisterPage = (req, res) => {
  res.render('register'); // Renderiza la vista de registro
};

// Función para manejar la solicitud de inicio de sesión
exports.loginUser = (req, res) => {
  // Lógica para autenticar al usuario y redirigirlo a la página principal
};

// Función para manejar la solicitud de registro de usuario
exports.registerUser = (req, res) => {
  // Lógica para registrar al usuario y redirigirlo a la página de inicio de sesión
};
```

4. Exportar los controladores: Al final del archivo, exporta las funciones del controlador para que estén disponibles para ser utilizadas en otras partes de tu aplicación:

```
module.exports = {
  getLoginPage,
  getRegisterPage,
  loginUser,
  registerUser
};
```

5. Usar los controladores en las rutas: En tu archivo principal de rutas (por lo general app.js o routes/index.js), importa los controladores y úsalos para manejar las solicitudes HTTP. Por ejemplo:

```
const express = require('express');
```

```
const router = express.Router();
const userController = require('../controllers/userController');

// Ruta para la página de inicio de sesión
router.get('/login', userController.getLoginPage);

// Ruta para la página de registro
router.get('/register', userController.getRegisterPage);

// Ruta para manejar el inicio de sesión del usuario
router.post('/login', userController.loginUser);

// Ruta para manejar el registro de usuario
router.post('/register', userController.registerUser);

module.exports = router;
```

Con estos pasos, has creado controladores básicos en tu aplicación Express y los has utilizado para manejar diferentes solicitudes HTTP. Esto te ayudará a mantener tu código organizado y modular, lo que facilitará el mantenimiento y la escalabilidad de tu aplicación.

Creación de vistas

La creación de vistas en una aplicación web con Express implica el diseño y la presentación de la interfaz de usuario que será mostrada al usuario final. En Express, las vistas se generan mediante el uso de motores de plantillas como Pug (anteriormente conocido como Jade), EJS (Embedded JavaScript), Handlebars, entre otros. A continuación, te mostraré cómo puedes crear vistas utilizando el motor de plantillas Pug como ejemplo:

1. Instalar el motor de plantillas Pug: Si aún no lo has hecho, instala el paquete pug utilizando npm:

```
npm install pug
```


2. Configurar Express para usar Pug como motor de plantillas: En tu archivo principal de la aplicación (por lo general app.js), configura Express para usar Pug como motor de plantillas:

```
const express = require('express');
const app = express();

// Configurar Express para usar Pug como motor de plantillas
app.set('view engine', 'pug');
app.set('views', path.join(__dirname, 'views')); // Directorio donde
se encuentran las vistas
```

3. Crear una vista en Pug: Dentro del directorio de vistas que especificaste en el paso anterior, crea un archivo .pug para cada vista que desees crear. Por ejemplo, podrías tener un archivo llamado index.pug para la página de inicio de tu aplicación:

```
// views/index.pug

doctype html
html
  head
    title Mi Aplicación
  body
    h1 ¡Bienvenido a Mi Aplicación!
    p Este es un ejemplo de una vista en Pug.
```

4. Renderizar la vista desde Express: En tu controlador o en el archivo de rutas correspondiente, renderiza la vista utilizando res.render(). Por ejemplo, podrías renderizar la vista index.pug para la ruta de la página de inicio:

```
const express = require('express');
const router = express.Router();

// Ruta para la página de inicio
router.get('/', (req, res) => {
  res.render('index'); // Renderiza la vista index.pug
});

module.exports = router;
```

5. Enviar datos a la vista: Si necesitas pasar datos dinámicos a la vista, puedes hacerlo proporcionando un objeto como segundo argumento a `res.render()`. Por ejemplo:

```
// En tu controlador
router.get('/perfil', (req, res) => {
  const usuario = {
    nombre: 'Juan',
    edad: 30,
    email: 'juan@example.com'
  };
  res.render('perfil', { usuario }); // Renderiza la vista
  perfil.pug y pasa el objeto usuario
});
```

```
// views/perfil.pug

doctype html
html
  head
    title Perfil de #{usuario.nombre}
  body
    h1 Perfil de #{usuario.nombre}
    p Nombre: #{usuario.nombre}
    p Edad: #{usuario.edad}
    p Email: #{usuario.email}
```

Con estos pasos, has creado una vista básica utilizando el motor de plantillas Pug en tu aplicación Express. Ahora puedes continuar diseñando y creando las diferentes vistas necesarias para tu aplicación web. Recuerda que puedes utilizar Pug, EJS u otros motores de plantillas según tus preferencias y requisitos del proyecto.

Creación de modelo de datos con MongoDB y Mongoose

Conectando aplicaciones en Express a MongoDB usando Mongoose

Para trabajar con MongoDB en una aplicación Express, Mongoose es una herramienta muy útil que proporciona una capa de modelado de datos y una interfaz para interactuar con la base de datos MongoDB de una manera más sencilla y estructurada. A continuación, te mostraré cómo puedes crear modelos de datos, definir esquemas y conectar tu aplicación Express a MongoDB utilizando Mongoose:

1. Instalar Mongoose: Si aún no lo has hecho, instala el paquete mongoose utilizando npm: `npm install mongoose`
2. Conexión a la base de datos MongoDB: En tu archivo principal de la aplicación (por lo general `app.js`), configura Express para conectarse a tu base de datos MongoDB utilizando Mongoose:

```
const mongoose = require('mongoose');

// URL de conexión a la base de datos MongoDB
const dbURI = 'mongodb://localhost:27017/nombre_base_de_datos';

// Conexión a la base de datos MongoDB
mongoose.connect(dbURI, { useNewUrlParser: true, useUnifiedTopology:
true })
  .then(() => console.log('Conexión exitosa a la base de datos
MongoDB'))
  .catch(err => console.error('Error al conectar a la base de
datos MongoDB:', err));
```

Modelado de datos

El modelado de datos en bases de datos NoSQL, como MongoDB, difiere del enfoque tradicional de bases de datos relacionales. En MongoDB, se utiliza un modelo de datos flexible y escalable que se basa en documentos JSON (BSON en realidad), en lugar de filas y tablas como en las bases de datos relacionales. A continuación, explicaré los conceptos clave del modelado de datos en MongoDB y proporcionaré ejemplos preparados para ilustrarlos:

Documentos: En MongoDB, los datos se almacenan en documentos BSON (Binary JSON), que son estructuras de datos flexibles y jerárquicas. Cada documento es una colección de pares clave-valor, similar a un objeto JSON. Por ejemplo, aquí hay un documento que representa un usuario:

```
{
  "_id": ObjectId("60932b9f2417bf1f3c87cd3a"),
  "nombre": "Juan",
  "edad": 30,
  "email": "juan@example.com"
}
```

Colecciones: Los documentos se agrupan en colecciones, que son equivalentes a tablas en bases de datos relacionales. Sin embargo, a diferencia de las tablas, las colecciones en MongoDB no tienen un esquema fijo, lo que permite una flexibilidad significativa en la estructura de los datos. Por ejemplo, una colección de usuarios podría contener documentos de usuarios como el mostrado anteriormente.

Esquemas dinámicos: MongoDB admite esquemas dinámicos, lo que significa que no es necesario definir una estructura de esquema fija antes de insertar datos en la base de datos. Cada documento en una colección puede tener una estructura diferente, lo que permite adaptarse fácilmente a cambios en los requisitos de la aplicación. Por ejemplo, podrías tener documentos de usuario con diferentes conjuntos de campos:

```
{
  "_id": ObjectId("60932b9f2417bf1f3c87cd3a"),
  "nombre": "Juan",
  "edad": 30,
  "email": "juan@example.com"
}
{
  "_id": ObjectId("60932b9f2417bf1f3c87cd3b"),
  "nombre": "María",
  "email": "maria@example.com",
  "direccion": "Calle Principal 123"
}
```

Referencias y embebidos: En MongoDB, puedes modelar relaciones entre datos utilizando referencias o embebidos. Las referencias son enlaces a documentos en otras colecciones, mientras que los embebidos incrustan documentos directamente dentro de otros documentos. La elección entre referencias y embebidos depende de los patrones de acceso y consulta de datos de tu aplicación. Por ejemplo, en un modelo de blog, podrías embeber los comentarios dentro de las entradas del blog o referenciarlos en una colección separada.

```
// Embebido
{
  "_id": ObjectId("60932b9f2417bf1f3c87cd3c"),
  "titulo": "Mi primera entrada de blog",
  "contenido": "¡Hola, mundo!",
  "comentarios": [
    {
      "usuario": "Ana",
      "contenido": "¡Genial!"
    },
    {
      "usuario": "Pedro",
      "contenido": "Excelente artículo."
    }
  ]
}

// Referencia
{
  "_id": ObjectId("60932b9f2417bf1f3c87cd3d"),
  "titulo": "Mi segunda entrada de blog",
  "contenido": "¡Hola de nuevo!",
  "comentarios": [
    ObjectId("60932b9f2417bf1f3c87cd3e"), // Referencia al comentario 1
    ObjectId("60932b9f2417bf1f3c87cd3f") // Referencia al comentario 2
  ]
}
```

Estos son algunos de los conceptos fundamentales del modelado de datos en MongoDB. Al utilizar MongoDB, es importante comprender estos conceptos para diseñar una estructura de datos eficiente y escalable que se adapte a las necesidades de tu aplicación. La flexibilidad y la capacidad de escalar horizontalmente hacen que MongoDB sea una opción poderosa para una amplia gama de aplicaciones.

Definiendo esquemas con Mongoose

3. Definir un esquema con Mongoose: Antes de crear modelos de datos, necesitas definir un esquema que describa la estructura de tus documentos en la base de datos MongoDB. Puedes hacerlo utilizando la API de esquemas de Mongoose. Por ejemplo, podrías definir un esquema para un modelo de usuario:

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

// Definir el esquema para el modelo de usuario
const userSchema = new Schema({
  nombre: String,
  email: String,
  edad: Number
});

// Exportar el esquema
module.exports = mongoose.model('User', userSchema);
```

4. Usar el modelo en tu aplicación: Una vez que has definido un esquema, puedes utilizarlo para crear modelos de datos y realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) en tu base de datos MongoDB. Por ejemplo, podrías utilizar el modelo de usuario en un controlador para guardar un nuevo usuario en la base de datos:

```
const User = require('../models/user');

// Crear un nuevo usuario
const newUser = new User({
  nombre: 'Juan',
  email: 'juan@example.com',
  edad: 30
});

// Guardar el nuevo usuario en la base de datos
newUser.save()
  .then(() => console.log('Usuario creado exitosamente'))
  .catch(err => console.error('Error al crear usuario:', err));
```

5. Uso de la Shell de MongoDB: Además de utilizar Mongoose en tu aplicación Express, también puedes interactuar directamente con la base de datos MongoDB utilizando la shell de MongoDB. Puedes ejecutar la shell de MongoDB desde la línea de comandos utilizando el comando `mongo`. Desde la shell de MongoDB, puedes crear bases de datos, colecciones y realizar consultas y operaciones de actualización en tu base de datos MongoDB.

Uso de la Shell de MongoDB para crear bases de datos y manipular información

La shell de MongoDB es una interfaz de línea de comandos que te permite interactuar directamente con tu servidor de MongoDB. Con la shell de MongoDB, puedes crear bases de datos, colecciones, insertar y consultar datos, entre otras operaciones. A continuación, te mostraré cómo puedes utilizar la shell de MongoDB para crear bases de datos, manipular información y realizar algunas operaciones básicas:

1. Iniciar la Shell de MongoDB: Para iniciar la shell de MongoDB, abre una terminal y ejecuta el siguiente comando: `mongo`
Esto te conectará a tu servidor de MongoDB en localhost utilizando el puerto predeterminado (27017) y te dará acceso a la shell de MongoDB.
2. Crear una base de datos: Para crear una nueva base de datos, simplemente especifica su nombre como parte de una operación de inserción o consulta. MongoDB creará la base de datos automáticamente cuando insertes el primer documento en ella. Por ejemplo, para crear una base de datos llamada `mi_base_de_datos`, puedes ejecutar: `use mi_base_de_datos`
Esto seleccionará la base de datos `mi_base_de_datos` como la base de datos activa en la sesión actual.
3. Crear una colección: Para crear una nueva colección dentro de la base de datos actual, puedes utilizar el método `db.createCollection()`. Por ejemplo,

para crear una colección llamada usuarios, puedes ejecutar:

```
db.createCollection("usuarios")
```

Esto creará una colección llamada usuarios dentro de la base de datos actual.

4. Insertar datos: Para insertar datos en una colección, puedes utilizar el método `db.collection.insertOne()` o `db.collection.insertMany()`. Por ejemplo, para insertar un nuevo usuario en la colección usuarios, puedes ejecutar:

```
db.usuarios.insertOne({nombre: "Juan", edad: 30})
```

Esto insertará un documento con el nombre "Juan" y la edad 30 en la colección usuarios.

5. Consultar datos: Para consultar datos en una colección, puedes utilizar el método `db.collection.find()`. Por ejemplo, para encontrar todos los usuarios en la colección usuarios, puedes ejecutar: `db.usuarios.find()`

Esto devolverá todos los documentos en la colección usuarios.

6. Actualizar datos: Para actualizar datos en una colección, puedes utilizar el método `db.collection.updateOne()` o `db.collection.updateMany()`. Por ejemplo, para actualizar el nombre de un usuario en la colección usuarios, puedes ejecutar:

```
db.usuarios.updateOne({nombre: "Juan"}, {$set: {nombre: "Pedro"}})
```

Esto actualizará el nombre del usuario de "Juan" a "Pedro".

7. Estos son solo algunos ejemplos básicos de cómo puedes utilizar la shell de MongoDB para crear bases de datos, colecciones e insertar y consultar datos. La shell de MongoDB proporciona una amplia gama de funcionalidades y operaciones que te permiten administrar y manipular tu base de datos MongoDB de manera eficiente y flexible.

Creación de APIs REST para exponer bases de datos de MongoDB a la aplicación

Configurando APIs en Express

Para crear una API REST que exponga los datos de una base de datos MongoDB en tu aplicación Express, necesitarás configurar rutas en Express y definir controladores para manejar las solicitudes HTTP utilizando los métodos GET, POST, PUT y DELETE. A continuación, te mostraré cómo puedes configurar una API REST en Express para interactuar con una base de datos MongoDB:

1. Instalar Mongoose: Si aún no lo has hecho, asegúrate de tener Mongoose instalado en tu proyecto de Express: `npm install mongoose`
2. Definir un modelo de datos con Mongoose: Define un esquema y un modelo de datos utilizando Mongoose para representar los documentos en tu base de datos MongoDB. Por ejemplo, podrías tener un modelo para usuarios:

```
// models/User.js

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  nombre: String,
  email: String,
  edad: Number
});

module.exports = mongoose.model('User', userSchema);
```

Métodos GET, POST, PUT, DELETE

3. Configurar rutas en Express para la API: En tu archivo de rutas de Express, configura las rutas para manejar las solicitudes HTTP para la API. Por ejemplo, podrías tener rutas para obtener todos los usuarios, crear un nuevo usuario, actualizar un usuario existente y eliminar un usuario:

```

// routes/api.js

const express = require('express');
const router = express.Router();
const User = require('../models/User');

// Obtener todos los usuarios
router.get('/users', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// Crear un nuevo usuario
router.post('/users', async (req, res) => {
  const user = new User({
    nombre: req.body.nombre,
    email: req.body.email,
    edad: req.body.edad
  });

  try {
    const newUser = await user.save();
    res.status(201).json(newUser);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

// Actualizar un usuario existente
router.put('/users/:id', async (req, res) => {
  const { id } = req.params;

  try {
    const user = await User.findById(id);
    if (!user) return res.status(404).json({ message: 'Usuario no encontrado' });

    user.nombre = req.body.nombre || user.nombre;
    user.email = req.body.email || user.email;
    user.edad = req.body.edad || user.edad;

    const updatedUser = await user.save();
    res.json(updatedUser);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});

// Eliminar un usuario
router.delete('/users/:id', async (req, res) => {
  const { id } = req.params;

```

```

    try {
      const user = await User.findById(id);
      if (!user) return res.status(404).json({ message: 'Usuario
no encontrado' });

      await user.remove();
      res.json({ message: 'Usuario eliminado' });
    } catch (error) {
      res.status(500).json({ message: error.message });
    }
  });
});

module.exports = router;

```

4. Agregar middleware para manejar solicitudes JSON: Asegúrate de agregar middleware a tu aplicación Express para manejar solicitudes y respuestas JSON. Esto permitirá que tu API REST pueda recibir datos en formato JSON y enviar respuestas JSON:

```

const express = require('express');
const app = express();

app.use(express.json());

```

Con estos pasos, has configurado una API REST en Express para exponer los datos de una base de datos MongoDB. Ahora puedes realizar solicitudes HTTP a tu API utilizando los métodos GET, POST, PUT y DELETE para interactuar con los datos en tu base de datos. Recuerda manejar adecuadamente los errores y las validaciones de datos para garantizar la robustez y la seguridad de tu API.

Consumo de APIs REST de Express

Invocar APIs desde Express

Para invocar APIs desde Express, puedes utilizar módulos como axios, node-fetch, o el propio módulo http de Node.js. Estos módulos te permiten hacer solicitudes HTTP a otras APIs desde tu servidor Express. A continuación, te mostraré cómo puedes usar axios para invocar una API desde Express:

1. Instalar axios: Si aún no lo has hecho, instala el módulo axios utilizando npm:

```
npm install axios
```

2. Utilizar axios en tu aplicación Express: Puedes utilizar axios en tus controladores o en cualquier parte de tu aplicación Express donde necesites hacer solicitudes a una API externa. Aquí tienes un ejemplo de cómo podrías hacer una solicitud GET a una API externa desde un controlador en Express:

```
const express = require('express');
const axios = require('axios');
const router = express.Router();

// Ruta para obtener datos de una API externa
router.get('/data', async (req, res) => {
  try {
    // Hacer una solicitud GET a la API externa
    const response = await
    axios.get('https://api.example.com/data');

    // Devolver los datos de la API externa como respuesta
    res.json(response.data);
  } catch (error) {
    // Manejar errores
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

3. Manejar otros métodos HTTP: Puedes utilizar axios para hacer solicitudes HTTP con otros métodos como POST, PUT o DELETE de manera similar. Por ejemplo:

```
// Hacer una solicitud POST a la API externa
const response = await axios.post('https://api.example.com/data',
requestData);

// Hacer una solicitud PUT a la API externa
const response = await axios.put('https://api.example.com/data/123',
updateData);

// Hacer una solicitud DELETE a la API externa
const response = await
axios.delete('https://api.example.com/data/123');
```

4. Manejar datos y errores: Recuerda manejar adecuadamente los datos devueltos por la API externa y los posibles errores que puedan ocurrir durante la solicitud. Puedes utilizar bloques try/catch para capturar errores y devolver respuestas apropiadas en tu aplicación Express.

Con estos pasos, has aprendido a utilizar axios para hacer solicitudes a APIs externas desde tu aplicación Express. Ahora puedes integrar fácilmente datos de otras fuentes en tu aplicación y construir servicios web más robustos y completos

Use de datos obtenidos desde APIs

Una vez que has obtenido datos de una API externa en tu aplicación Express, puedes utilizar esos datos de diversas maneras según los requisitos de tu aplicación. Aquí hay algunas formas comunes de utilizar los datos obtenidos desde APIs en tu aplicación:

1. Renderizar datos en vistas: Puedes pasar los datos obtenidos desde la API a tus vistas y renderizarlos para que sean mostrados al usuario. Por ejemplo, podrías renderizar una lista de elementos en una página web utilizando un motor de plantillas como Pug o EJS:

```
// Controlador en Express
const express = require('express');
const axios = require('axios');
const router = express.Router();

// Ruta para obtener datos de una API externa
router.get('/data', async (req, res) => {
  try {
    // Hacer una solicitud GET a la API externa
    const response = await
    axios.get('https://api.example.com/data');

    // Renderizar la vista y pasar los datos obtenidos como
    contexto
    res.render('pagina', { datos: response.data });
  } catch (error) {
    // Manejar errores
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

```
// Vista en Pug
doctype html
html
  head
    title Datos de la API
  body
    h1 Datos de la API Externa
    ul
      each item in datos
        li= item
```

2. Manipular y procesar datos: Puedes manipular y procesar los datos obtenidos desde la API antes de enviarlos a tus vistas o almacenarlos en tu base de datos. Por ejemplo, podrías filtrar, ordenar o transformar los datos según tus necesidades específicas antes de mostrarlos al usuario.
3. Almacenar datos en tu base de datos: Si necesitas almacenar los datos obtenidos desde la API externa para su posterior uso en tu aplicación, puedes guardarlos en tu base de datos utilizando un modelo de datos de Mongoose. Por ejemplo, podrías guardar los datos en una colección de MongoDB:

```

const express = require('express');
const axios = require('axios');
const router = express.Router();
const Datos = require('../models/Datos');

router.get('/data', async (req, res) => {
  try {
    const response = await
    axios.get('https://api.example.com/data');

    // Guardar los datos en la base de datos
    const nuevosDatos = new Datos({
      datos: response.data
    });
    await nuevosDatos.save();

    res.json(response.data);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;

```

Recuerda que el uso específico de los datos obtenidos desde una API dependerá de los requisitos de tu aplicación y del flujo de trabajo de tu aplicación. Siempre asegúrate de manejar adecuadamente los errores y de validar los datos antes de usarlos para garantizar la robustez y la seguridad de tu aplicación.

Obtener documentos usando APIs

Si deseas obtener documentos específicos de una API externa en tu aplicación Express, puedes hacerlo utilizando solicitudes HTTP con el método GET y proporcionando parámetros de consulta para especificar qué documentos deseas recuperar. A continuación, te mostraré cómo puedes obtener documentos específicos utilizando la API desde Express:

```

const express = require('express');
const axios = require('axios');
const router = express.Router();

// Ruta para obtener un documento específico de la API externa
router.get('/documents/:id', async (req, res) => {

```

```

const { id } = req.params;

try {
  // Hacer una solicitud GET a la API externa con el ID del
  documento
  const response = await
  axios.get(`https://api.example.com/documents/${id}`);

  // Verificar si se encontró el documento
  if (response.status === 200) {
    // Devolver el documento encontrado
    res.json(response.data);
  } else {
    // Si el documento no se encuentra, devolver un error
    404
    res.status(404).json({ message: 'Documento no
    encontrado' });
  }
} catch (error) {
  // Manejar errores de la solicitud
  res.status(500).json({ message: error.message });
}
});

module.exports = router;

```

En este ejemplo, hemos configurado una ruta en Express que espera un parámetro `id` en la URL para especificar el documento que deseamos recuperar de la API externa. Luego, hacemos una solicitud GET a la API externa incluyendo este ID en la URL de la solicitud. Si la solicitud es exitosa (es decir, si el documento se encuentra), devolvemos el documento como respuesta. Si el documento no se encuentra, devolvemos un error 404. Si ocurre algún error durante la solicitud, devolvemos un error 500.

Puedes usar esta ruta en tu aplicación Express para obtener documentos específicos de una API externa, pasando el ID del documento deseado como parte de la URL. Asegúrate de manejar adecuadamente los casos de error y de validar los datos recibidos para garantizar la robustez y la seguridad de tu aplicación.

Agregando datos a la base de datos por medio de APIs

Si deseas agregar datos a tu base de datos MongoDB utilizando una API en tu aplicación Express, puedes hacerlo utilizando solicitudes HTTP con los métodos POST. A continuación, te mostraré cómo puedes agregar datos a tu base de datos MongoDB mediante una solicitud POST a una ruta específica en tu aplicación Express:

1. Configurar la ruta en Express para agregar datos:

```
const express = require('express');
const router = express.Router();
const mongoose = require('mongoose');
const User = require('../models/User'); // Suponiendo que tienes un
modelo de datos para usuarios

// Ruta para agregar un nuevo usuario a la base de datos
router.post('/users', async (req, res) => {
  try {
    // Crear un nuevo objeto de usuario utilizando los datos
    // proporcionados en el cuerpo de la solicitud
    const newUser = new User({
      nombre: req.body.nombre,
      email: req.body.email,
      edad: req.body.edad
    });

    // Guardar el nuevo usuario en la base de datos
    const usuarioGuardado = await newUser.save();

    // Devolver una respuesta con el usuario recién creado
    res.status(201).json(usuarioGuardado);
  } catch (error) {
    // Manejar errores
    res.status(400).json({ message: error.message });
  }
});

module.exports = router;
```

2. Hacer una solicitud POST a la API desde el cliente:

Desde el cliente o cualquier otra fuente que desees utilizar para enviar datos a tu API, puedes hacer una solicitud POST a la ruta que configuraste en tu aplicación Express. Asegúrate de enviar los datos en el cuerpo de la solicitud en el formato adecuado, generalmente en formato JSON.

Por ejemplo, si estás utilizando JavaScript en el cliente (por ejemplo, en un navegador web), puedes hacer una solicitud POST utilizando fetch de la siguiente manera:

```
fetch('/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    nombre: 'Juan',
    email: 'juan@example.com',
    edad: 30
  })
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error('Error:', error));
```

En este ejemplo, estamos enviando un objeto con los datos del usuario en el cuerpo de la solicitud a la ruta /users de nuestra API Express. La API procesará esta solicitud y agregará un nuevo usuario a la base de datos MongoDB.

Al utilizar este enfoque, puedes agregar datos a tu base de datos MongoDB a través de una API en tu aplicación Express de manera fácil y segura. Asegúrate de manejar adecuadamente los errores y validar los datos recibidos para garantizar la integridad y la seguridad de tu base de datos.

Protección de la integridad de los datos con validaciones de datos

Para proteger la integridad de los datos en tu base de datos MongoDB mediante validaciones de datos, puedes utilizar tanto las validaciones proporcionadas por el propio MongoDB como también realizar validaciones personalizadas en tu aplicación Express utilizando bibliotecas de validación como express-validator o validaciones manuales en tus controladores. A continuación, te mostraré cómo puedes implementar diferentes formas de validación de datos en tu aplicación Express:

1. Validaciones en MongoDB: MongoDB ofrece la posibilidad de definir reglas de validación a nivel de esquema para tus colecciones utilizando validadores integrados o expresiones regulares. Por ejemplo, podrías definir un esquema de validación en Mongoose que exija que el campo de correo electrónico sea único:

```
const userSchema = new mongoose.Schema({
  nombre: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true // Validación de unicidad
  },
  edad: {
    type: Number,
    min: 18 // Edad mínima
  }
});
```

2. Validaciones en Express con express-validator: Puedes usar la biblioteca express-validator para definir y aplicar reglas de validación en tus rutas de Express. Por ejemplo, podrías definir una ruta que valide los datos de un nuevo usuario antes de agregarlo a la base de datos:

```
const { body, validationResult } = require('express-validator');

router.post('/users', [
  // Validar nombre, email y edad
  body('nombre').notEmpty(),
  body('email').isEmail(),
  body('edad').isInt({ min: 18 })
], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  // Crear un nuevo usuario y guardarlo en la base de datos
  // ...
});
```

3. Validaciones manuales en controladores: También puedes realizar validaciones manualmente en tus controladores utilizando condiciones if para

verificar los datos antes de procesarlos. Por ejemplo, podrías verificar si se proporcionan datos de usuario válidos antes de agregarlos a la base de datos:

```
router.post('/users', (req, res) => {
  const { nombre, email, edad } = req.body;

  // Validar los datos del usuario
  if (!nombre || !email || !edad) {
    return res.status(400).json({ message: 'Se requieren todos
los campos: nombre, email, edad' });
  }
  if (edad < 18) {
    return res.status(400).json({ message: 'La edad debe ser
mayor de 18 años' });
  }

  // Crear un nuevo usuario y guardarlo en la base de datos
  // ...
});
```

Estos son solo algunos ejemplos de cómo puedes implementar validaciones de datos en tu aplicación Express para proteger la integridad de tus datos en la base de datos MongoDB. Puedes elegir la opción que mejor se adapte a tus necesidades y requisitos específicos. Recuerda siempre validar los datos recibidos antes de procesarlos para garantizar la integridad y la seguridad de tu aplicación.

Bibliografía y Webgrafía

Arias, G. Node.js desde cero: Proyecto completo paso a paso. Lulu.com (30 Julio 2022)

Gómez, M. Clean JavaScript: Aprende a aplicar Código Limpio, SOLID y Testing. Independently published (10 Abril 2022)

Morales, G. Creando API con Node.js, express y MongoDB. Independently published (31 Agosto 2020)

- **API** (Interfaz de Programación de Aplicaciones): Conjunto de reglas y herramientas que permite la comunicación entre diferentes aplicaciones de software.
- **BSON** (Binary JSON): Formato de datos binarios similar a JSON utilizado en MongoDB para representar documentos.
- **Controlador**: Componente de una aplicación web que maneja las solicitudes del cliente y controla la lógica de negocio, incluyendo la interacción con la base de datos.
- **Documento**: Unidad básica de datos en MongoDB, representado en formato BSON y almacenado en una colección.
- **Esquema**: Una estructura que define la forma y las restricciones de los documentos en una colección en MongoDB.
- **Express**: Framework de aplicaciones web para Node.js que proporciona un conjunto de características para la construcción de aplicaciones web y APIs.
- **GET**: Método HTTP utilizado para solicitar datos de un recurso específico.
- **JSON** (JavaScript Object Notation): Formato de intercambio de datos ligero y legible por humanos, basado en texto, utilizado para representar datos estructurados.
- **Mongoose**: Biblioteca de modelado de datos para MongoDB en Node.js que proporciona una solución de modelado de objetos y esquemas para gestionar la conexión a la base de datos y definir modelos de datos.
- **MVC** (Modelo-Vista-Controlador): Patrón de diseño arquitectónico que separa la lógica de presentación y la lógica de negocio en una aplicación, dividiéndola en tres componentes principales: modelo, vista y controlador.
- **POST**: Método HTTP utilizado para enviar datos para ser procesados por un recurso específico.
- **PUT**: Método HTTP utilizado para actualizar datos de un recurso específico.

- **REST** (Transferencia de Estado Representacional): Estilo de arquitectura de software que define un conjunto de restricciones para la creación de servicios web, como API RESTful.
- **Shell de MongoDB**: Interfaz de línea de comandos utilizada para interactuar con una instancia de MongoDB.
- **Validación de datos**: Proceso de asegurar que los datos ingresados cumplen con ciertos criterios especificados antes de ser procesados o almacenados.